PROPOSAL PROYEK AKHIR

PERBANDINGAN ALGORITMA KLASIFIKASI UNTUK MEMPREDIKSI PELUANG MENDAPATKAN S-RANK PADA SISTEM GACHA ZENLESS ZONE ZERO BERBASIS WEB

Disusun untuk memenuhi Ujian Tengah Semester (UTS)
mata kuliah **Penambangan Data**

Kelas **A11.4502** · Kelompok **6**

NIM	Nama	Peran
A11.2024.16004	Syafiq Yahya	Ketua kelompok, presentasi hasil
A11.2024.15851	Dafi Hauzan A.H	Pengembang aplikasi, pengumpulan dan pengujian data
A11.2024.15842	Gastiadirrijal Rafi M	Penyusun slide presentasi
A11.2024.15826	Zabrina Miftah Z	Penyusun proposal dan laporan
A11.2024.15804	Rayya Hasya Tamimi	Penyusun proposal dan laporan

Dosen Pengampu:
Ardytha Luthfiarta, M.Kom, MCS

PROGRAM STUDI TEKNIK INFORMATIKA – S1
FAKULTAS ILMU KOMPUTER
UNIVERSITAS DIAN NUSWANTORO
SEMARANG

2026

KATA PENGANTAR

Puji syukur kami panjatkan kepada Tuhan Yang Maha Esa karena atas rahmat-Nya kami dapat menyelesaikan proposal proyek akhir berjudul "Perbandingan algoritma klasifikasi untuk memprediksi peluang mendapatkan S-Rank pada sistem gacha Zenless Zone Zero berbasis web". Dokumen ini disusun untuk memenuhi Ujian Tengah Semester mata kuliah Penambangan Data.

Kami mengucapkan terima kasih kepada Bapak Ardytha Luthfiarta, M.Kom, MCS selaku dosen pengampu yang telah memberikan materi dan arahan, serta kepada teman-teman pemain Zenless Zone Zero yang bersedia menyumbangkan riwayat gacha mereka secara anonim sebagai data penelitian.

Kami menyadari dokumen ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan. Semoga dokumen ini bermanfaat bagi pembaca.

Semarang, Oktober 2026
Kelompok 6

DAFTAR ISI

KATA PENGANTAR.....ii
DAFTAR TABEL.....v
DAFTAR GAMBAR.....vi
BAB I PENDAHULUAN.....1
1.1 Latar Belakang.....1
1.2 Alasan Menggunakan Penambangan Data.....2
1.3 Rumusan Masalah.....2
1.4 Batasan Masalah.....3
1.5 Tujuan.....3
1.6 Manfaat.....3
BAB II TINJAUAN PUSTAKA.....5
2.1 Penambangan Data dan CRISP-DM.....5
2.2 Klasifikasi (Supervised Learning).....5
2.3 Sistem Gacha dan Pity pada Zenless Zone Zero.....5
2.4 Markov Chain.....6
2.5 Algoritma Klasifikasi.....7
2.5.1 Naive Bayes.....7
2.5.2 K-Nearest Neighbor (KNN).....7
2.5.3 Logistic Regression.....8
2.5.4 Decision Tree.....8
2.5.5 Random Forest.....8
2.5.6 XGBoost.....8
2.6 Evaluasi Model.....8
2.7 Pemilihan Hyperparameter dan Uji Signifikansi.....9
2.8 Streamlit.....10
BAB III METODE PENELITIAN.....11
3.1 Alur Penelitian.....11
3.2 Sumber Data.....12
3.3 Pra-pemrosesan Data.....12
3.4 Fitur dan Target.....13
3.5 Pemodelan.....13
3.6 Skenario Evaluasi.....14
3.7 Rancangan Aplikasi.....15
3.8 Alat dan Bahan.....16
3.9 Hasil Awal (Prototipe).....16
3.10 Jadwal Kegiatan.....18
3.11 Pembagian Tugas.....18

DAFTAR PUSTAKA..... 19

DAFTAR TABEL

Tabel 1.1 Pemeriksaan perlu tidaknya machine learning.....2
Tabel 2.1 Parameter banner terbatas Zenless Zone Zero.....6
Tabel 2.2 Metrik evaluasi yang digunakan.....9
Tabel 3.1 Sumber data penelitian.....12
Tabel 3.2 Fitur yang digunakan.....13
Tabel 3.3 Algoritma, pengaturan tetap, dan ruang pencarian grid search.....14
Tabel 3.4 Halaman aplikasi.....16
Tabel 3.5 Jadwal kegiatan proyek.....18
Tabel 3.6 Pembagian tugas anggota kelompok.....18

DAFTAR GAMBAR

Gambar 2.1 Peluang S-Rank per pull (hazard) untuk kedua banner.....7
Gambar 2.2 Ilustrasi rantai Markov sistem pity.....7
Gambar 3.1 Alur penelitian berdasarkan CRISP-DM.....11
Gambar 3.2 Prototipe halaman Kalkulator Peluang.....17

BAB I
PENDAHULUAN

1.1 Latar Belakang

Gacha adalah mekanisme undian berhadiah di dalam permainan digital. Pemain menukarkan mata uang permainan untuk satu kali undian (*pull*) dan mendapatkan item secara acak sesuai peluang yang ditetapkan pengembang. Mekanisme ini banyak dipakai pada permainan *free-to-play* modern, salah satunya **Zenless Zone Zero** (ZZZ) yang dirilis HoYoverse pada tahun 2024.

Pada ZZZ, hadiah paling langka adalah item **S-Rank**, baik karakter (*Agent*) maupun senjata (*W-Engine*). Peluang dasarnya kecil, yaitu 0,6% untuk Agent dan 1,0% untuk W-Engine pada banner terbatas. Untuk mencegah pemain terlalu lama tidak beruntung, permainan memakai sistem **pity**: semakin lama pemain tidak mendapatkan S-Rank, peluangnya meningkat (*soft pity*) hingga pada batas tertentu S-Rank pasti didapat (*hard pity*, pull ke-90 untuk Agent dan ke-80 untuk W-Engine). Selain itu terdapat aturan 50/50 (Agent) dan 75/25 (W-Engine) yang menentukan apakah S-Rank yang keluar adalah item yang sedang dipromosikan (*rate-up*).

Kombinasi aturan tersebut membuat pemain sulit memperkirakan sendiri peluang mendapatkan S-Rank, misalnya pertanyaan "jika pity saya sekarang 70, berapa peluang saya mendapatkan S-Rank dalam 10 pull berikutnya?". Padahal, riwayat gacha setiap pemain tersimpan di server dan dapat diambil sebagai data. Data riwayat ini berpotensi diolah dengan teknik penambangan data untuk menjawab pertanyaan tersebut secara kuantitatif.

Penambangan data menyediakan banyak algoritma klasifikasi, di antaranya Naive Bayes, K-Nearest Neighbor (KNN), Logistic Regression, Decision Tree, Random Forest, dan XGBoost. Masing-masing memiliki asumsi dan cara belajar yang berbeda, sehingga kinerjanya perlu dibandingkan pada kasus yang sama. Kasus gacha juga memiliki tantangan khusus, yaitu **data yang sangat tidak seimbang**: kelas "mendapat S-Rank" hanya sekitar 1,7% dari seluruh pull, sehingga metrik akurasi saja dapat menyesatkan.

Berdasarkan hal tersebut, proyek ini akan membangun aplikasi berbasis web yang dapat diakses dari ponsel maupun komputer untuk menghitung peluang mendapatkan S-Rank dengan model Markov Chain, sekaligus membandingkan kinerja enam algoritma klasifikasi dalam memprediksi keluarnya S-Rank.

1.2 Alasan Menggunakan Penambangan Data

Sebelum memilih algoritma, perlu dipastikan bahwa kasus ini memang membutuhkan penambangan data. *Machine learning* tidak diperlukan apabila (1) hasil dapat ditentukan langsung oleh suatu atribut, atau (2) sudah tersedia pengetahuan atau rumus yang pasti dan lengkap. Tabel 1.1 menunjukkan bahwa kedua kondisi tersebut tidak terpenuhi pada sistem gacha Zenless Zone Zero.

Tabel 1.1 Pemeriksaan perlu tidaknya machine learning

Kondisi	**Pada gacha Zenless Zone Zero**	**Terpenuhi?**
Ada atribut yang langsung menentukan hasil	Tidak ada. Pity hanya mengubah peluang; hasil setiap pull tetap acak. Kepastian hanya terjadi di hard pity.	Tidak
Ada rumus yang pasti dan lengkap	Hanya sebagian. Rate dasar, rate gabungan, hard pity, dan peluang rate-up diumumkan resmi, tetapi kapan soft pity dimulai dan seberapa cepat peluang naik tidak diumumkan. Perkiraan komunitas pun berbeda-beda.	Tidak

Bagian aturan yang tidak diumumkan tersebut hanya dapat diketahui dari data riwayat gacha pemain. Karena setiap pull memiliki label yang jelas (mendapat S-Rank atau tidak), masalah ini termasuk **klasifikasi** sekaligus **estimasi** peluang dalam *supervised learning*. Model Markov Chain tetap akan dipakai, tetapi sebagai **pembanding teori** yang dibangun dari parameter resmi ditambah estimasi soft pity. Algoritma klasifikasi tidak diberi tahu tentang soft pity, sehingga dapat diuji apakah mampu menemukan pola tersebut sendiri dari data.

1.3 Rumusan Masalah

1. Bagaimana menghitung peluang pemain mendapatkan S-Rank dan S-Rank *rate-up* dalam N pull berdasarkan pity dan status *guaranteed* pada banner terbatas Zenless Zone Zero?
2. Apakah pola kenaikan peluang (*soft pity*) yang tidak diumumkan resmi dapat ditemukan dari data riwayat gacha pemain?
3. Algoritma klasifikasi manakah di antara Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, dan XGBoost yang paling akurat memprediksi keluarnya S-Rank?
4. Bagaimana mengevaluasi model secara tepat pada data yang sangat tidak seimbang?

5. Bagaimana menyajikan hasil tersebut dalam aplikasi web yang mudah diakses pengguna ponsel maupun komputer?

1.4 Batasan Masalah

- Banner yang diteliti hanya **banner Agent terbatas** (*Exclusive Channel*) dan **banner W-Engine terbatas** (*W-Engine Channel*). Banner standar dan Bangboo tidak dibahas.
- Yang diprediksi adalah keluarnya item **S-Rank**; item A-Rank dan B-Rank tidak dimodelkan.
- Parameter resmi (rate dasar, rate gabungan, hard pity, peluang *rate-up*) diambil dari detail banner di dalam permainan. Titik mulai *soft pity* tidak diumumkan resmi, sehingga model Markov memakai estimasi komunitas yang kemudian diperiksa dengan data asli.
- Data asli berasal dari riwayat akun anggota kelompok dan teman yang bersedia, dalam bentuk anonim.
- Aplikasi dibangun dengan Python dan Streamlit, dan pengambilan riwayat otomatis hanya tersedia untuk permainan versi PC (Windows).

1.5 Tujuan

1. Membangun model Markov Chain untuk menghitung peluang mendapatkan S-Rank dan S-Rank *rate-up* berdasarkan parameter banner.
2. Menggali pola *soft pity* yang tidak diumumkan resmi dari data riwayat gacha pemain.
3. Membandingkan kinerja enam algoritma klasifikasi dalam memprediksi keluarnya S-Rank, dengan hyperparameter yang dipilih melalui *grid search* dan perbedaan kinerja yang diuji signifikansinya secara statistik.
4. Menerapkan metrik evaluasi yang sesuai untuk data tidak seimbang, yaitu Accuracy, Balanced Accuracy, Precision, Recall, F1-score, ROC-AUC, Log Loss, dan Brier Score.
5. Menyajikan seluruh hasil dalam aplikasi web yang dapat diakses dari ponsel, tablet, maupun komputer.

1.6 Manfaat

- **Bagi pemain:** mendapat gambaran peluang yang jelas untuk merencanakan penggunaan mata uang permainan.

- **Bagi mahasiswa:** menerapkan tahapan penambangan data (CRISP-DM), algoritma klasifikasi, evaluasi dengan *confusion matrix*, dan *deployment* dengan Streamlit sesuai materi perkuliahan.
- **Bagi keilmuan:** memberikan contoh penerapan penambangan data untuk menggali aturan yang tidak diumumkan (soft pity) dari data, sekaligus contoh nyata bahaya memakai akurasi sebagai satu-satunya metrik pada data tidak seimbang.

BAB II
TINJAUAN PUSTAKA

2.1 Penambangan Data dan CRISP-DM

Penambangan data (*data mining*) adalah proses menemukan pola, pengetahuan, dan informasi yang berguna dari kumpulan data berukuran besar (Han, Kamber, & Pei, 2012). Proses ini mengubah **data** mentah menjadi **informasi**, lalu menjadi **pengetahuan** yang dapat dipakai untuk mengambil keputusan.

Proyek ini mengikuti kerangka **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*) yang terdiri atas enam tahap: *business understanding*, *data understanding*, *data preparation*, *modeling*, *evaluation*, dan *deployment* (Chapman et al., 2000).

2.2 Klasifikasi (Supervised Learning)

Klasifikasi adalah tugas *supervised learning* untuk memetakan sekumpulan fitur ke salah satu label kelas berdasarkan contoh data berlabel. Pada proyek ini, label berupa dua kelas (klasifikasi biner): **mendapat S-Rank** atau **tidak mendapat S-Rank**. Selain label, sebagian besar algoritma juga menghasilkan **peluang** masuk ke tiap kelas. Peluang inilah yang membuat klasifikasi juga berperan sebagai **estimasi**, karena pemain tidak hanya membutuhkan tebakan ya atau tidak, tetapi juga seberapa besar peluangnya.

2.3 Sistem Gacha dan Pity pada Zenless Zone Zero

Fitur gacha pada Zenless Zone Zero disebut *Signal Search*. Setiap pull memiliki peluang tertentu menghasilkan item S-Rank. Parameter banner terbatas yang dipakai dalam proyek ini ditunjukkan pada Tabel 2.1. Angka resmi diambil dari menu detail *Signal Search* di dalam permainan (HoYoverse, 2024).

Tabel 2.1 Parameter banner terbatas Zenless Zone Zero

Parameter	Agent Terbatas	W-Engine Terbatas	Sumber
Rate dasar S-Rank	0,6%	1,0%	Resmi (detail banner)
Rate gabungan (termasuk pity)	1,6%	2,0%	Resmi (detail banner)
Hard pity	90	80	Resmi (detail banner)
Peluang S-Rank rate-up	50%	75%	Resmi (detail banner)
Soft pity mulai	pull ke-74	pull ke-65	Estimasi komunitas (tidak resmi)

Jika pemain mendapatkan S-Rank yang bukan *rate-up* (disebut kalah 50/50), maka S-Rank berikutnya **pasti** *rate-up*. Status ini disebut *guaranteed*. Riwayat gacha dapat diambil dari server resmi melalui URL yang memuat kunci akses sementara (*authkey*), dan dapat dipertukarkan antaraplikasi dengan format standar **UIGF** (*Uniformed Interchangeable GachaLog Format*) versi 4 (UIGF Organization, 2024).

2.4 Markov Chain

Markov Chain adalah proses stokastik ketika peluang berpindah ke keadaan (*state*) berikutnya hanya bergantung pada keadaan saat ini (Norris, 1997). Sistem pity cocok dimodelkan dengan Markov Chain karena peluang mendapatkan S-Rank pada suatu pull hanya bergantung pada jumlah pull sejak S-Rank terakhir (pity) dan status *guaranteed*.

Peluang mendapatkan S-Rank pada pull ke-n, dengan syarat belum mendapat S-Rank sebelumnya, disebut *hazard* h(n):

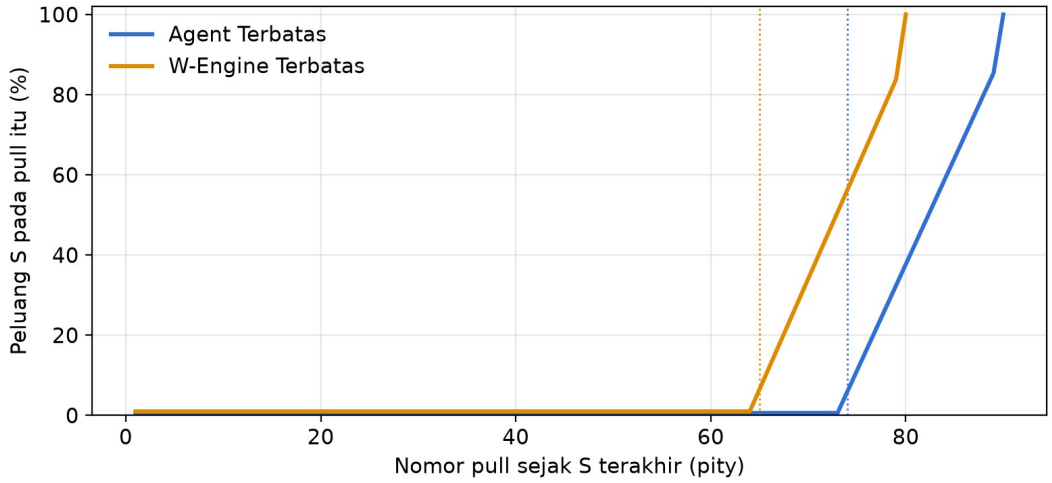
$$h(n) = p \qquad \text{untuk } n < s$$

$$h(n) = \min(1,\ p + k\cdot(n - s + 1)) \quad \text{untuk } n \geq s$$

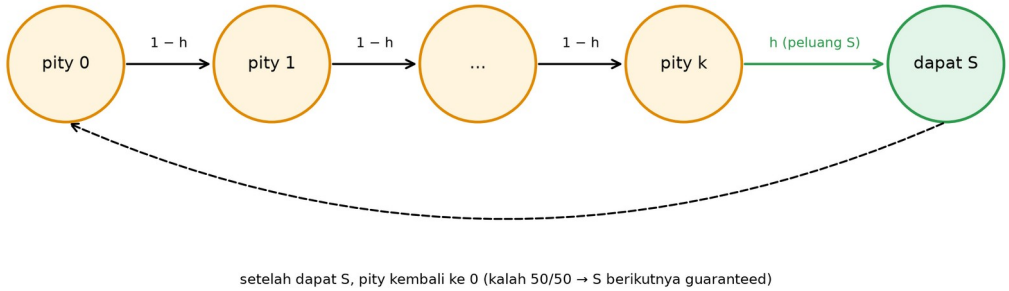
$$h(\text{hard pity}) = 1$$

dengan p = rate dasar, s = titik mulai soft pity, dan k = kenaikan peluang per pull. Karena nilai k tidak diumumkan, k dicari dengan **metode biseksi** sehingga rata-rata jumlah pull per S-Rank sama dengan kebalikan rate gabungan resmi (62,5 pull untuk Agent dan 50 pull untuk W-Engine). Hasilnya k ≈ 5,30% untuk Agent

dan k ≈ 5,52% untuk W-Engine. Nilai s (pull ke-74 dan ke-65) juga bukan angka resmi, melainkan estimasi komunitas yang diperiksa dengan data asli dalam penelitian ini.



Gambar 2.1 Peluang S-Rank per pull (hazard) untuk kedua banner



Gambar 2.2 Ilustrasi rantai Markov sistem pity

2.5 Algoritma Klasifikasi

2.5.1 Naive Bayes

Naive Bayes adalah pengklasifikasi probabilistik berdasarkan Teorema Bayes dengan asumsi bahwa setiap fitur saling bebas jika kelasnya diketahui. Varian Gaussian Naive Bayes mengasumsikan setiap fitur numerik berdistribusi normal di dalam setiap kelas. Algoritma ini sangat cepat, namun asumsinya sering tidak terpenuhi pada data nyata.

2.5.2 K-Nearest Neighbor (KNN)

KNN mengklasifikasikan data baru berdasarkan mayoritas label dari k data latih terdekat menurut jarak tertentu, umumnya jarak Euclidean (Cover & Hart,

1967). KNN tidak membangun model eksplisit (*lazy learner*), sehingga fitur perlu dinormalisasi agar tidak ada fitur yang mendominasi perhitungan jarak.

2.5.3 Logistic Regression

Logistic Regression memodelkan peluang kelas positif dengan fungsi sigmoid dari kombinasi linear fitur: $P(y = 1) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n)})$ (Hosmer, Lemeshow, & Sturdivant, 2013). Model ini mudah ditafsirkan, tetapi hanya mampu membentuk batas keputusan yang linear terhadap fitur.

2.5.4 Decision Tree

Decision Tree membagi data secara bertahap berdasarkan nilai fitur sehingga membentuk struktur pohon aturan *jika–maka* (Quinlan, 1986). Pembagian dipilih untuk memaksimalkan kemurnian kelas, misalnya dengan indeks Gini. Kelebihannya, aturan yang dihasilkan mudah dibaca manusia.

2.5.5 Random Forest

Random Forest adalah gabungan (*ensemble*) banyak Decision Tree yang masing-masing dilatih pada sampel acak data dan subset fitur acak, lalu hasilnya dirata-ratakan (*bagging*) (Breiman, 2001). Teknik ini mengurangi *overfitting* dibandingkan satu pohon.

2.5.6 XGBoost

XGBoost (*Extreme Gradient Boosting*) membangun pohon secara berurutan, di mana setiap pohon baru memperbaiki kesalahan pohon sebelumnya dengan optimasi gradien dan regularisasi (Chen & Guestrin, 2016). Algoritma ini dikenal kuat pada data tabular.

2.6 Evaluasi Model

Evaluasi klasifikasi biner didasarkan pada **confusion matrix** yang berisi True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN). Metrik yang dipakai dalam proyek ini ditunjukkan pada Tabel 2.2.

Tabel 2.2 Metrik evaluasi yang digunakan

Metrik	**Rumus / pengertian**
Accuracy	(TP + TN) / (TP + TN + FP + FN)
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
F1-score	2 × Precision × Recall / (Precision + Recall)
Balanced Accuracy	rata-rata recall kelas positif dan kelas negatif
ROC-AUC	luas di bawah kurva ROC; kemampuan membedakan kelas di semua ambang
Log Loss	−rata-rata [y·log(p) + (1−y)·log(1−p)]; menilai ketepatan angka peluang
Brier Score	rata-rata $(p - y)^2$; makin kecil, makin tepat angka peluangnya

Pada data tidak seimbang, akurasi dapat tinggi walaupun model tidak pernah menebak kelas minoritas (He & Garcia, 2009). Karena itu metrik seperti Balanced Accuracy, F1-score, dan ROC-AUC perlu digunakan sebagai pendamping (Saito & Rehmsmeier, 2015). Log Loss dan Brier Score (Brier, 1950) dipakai untuk menilai seberapa tepat peluang yang dihasilkan.

2.7 Pemilihan Hyperparameter dan Uji Signifikansi

Hyperparameter adalah pengaturan algoritma yang tidak dipelajari dari data, misalnya jumlah tetangga k pada KNN atau kedalaman maksimum pohon. *Grid search* mencoba setiap kombinasi nilai dari ruang pencarian yang ditentukan, lalu memilih kombinasi dengan skor cross-validation terbaik (Pedregosa et al., 2011). Data yang dipakai untuk memilih hyperparameter tidak boleh sama dengan data yang dipakai untuk menilai model akhir, karena skornya akan terlihat lebih baik daripada kenyataan. Jika data terbatas, dipakai *nested cross-validation*: grid search dijalankan hanya di dalam data latih setiap fold, sedangkan data uji fold tersebut hanya dipakai untuk penilaian (Varma & Simon, 2006).

Selisih akurasi antar-model juga perlu diuji, apakah bermakna secara statistik atau hanya kebetulan. Uji t berpasangan biasa pada hasil setiap fold terlalu mudah menyatakan dua model berbeda, karena data latih antarfold saling tumpang tindih sehingga hasilnya tidak saling bebas. Nadeau dan Bengio (2003) mengusulkan **uji t berpasangan terkoreksi** (*corrected resampled t-test*) yang memperbesar varians dengan faktor koreksi:

$$t = \bar{d} \,/\, \sqrt{\left(1/k + n_{uji} / n_{latih}\right) \cdot s^2_d}$$

dengan $\bar{d}$ dan s^2_d adalah rata-rata dan varians selisih akurasi dua model pada k fold, sedangkan n_uji/n_latih adalah perbandingan ukuran data uji dan data latih. Nilai t dibandingkan dengan distribusi t berderajat bebas $k - 1$. Jika p-value kurang dari 0,05, selisih kedua model dinyatakan **signifikan**.

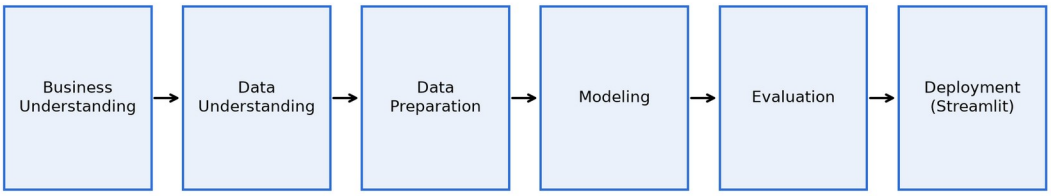
2.8 Streamlit

Streamlit adalah pustaka Python sumber terbuka untuk membangun aplikasi web interaktif bagi analisis data dan *machine learning* tanpa perlu menulis HTML, CSS, atau JavaScript (Streamlit Inc., 2024). Aplikasi Streamlit dapat dipublikasikan gratis melalui Streamlit Community Cloud sehingga dapat dibuka dari peramban ponsel maupun komputer.

BAB III
METODE PENELITIAN

3.1 Alur Penelitian

Penelitian akan dilaksanakan mengikuti tahapan CRISP-DM seperti pada Gambar 3.1.



Gambar 3.1 Alur penelitian berdasarkan CRISP-DM

1. **Business understanding:** merumuskan kebutuhan pemain, yaitu mengetahui peluang mendapatkan S-Rank dari pity saat ini.
2. **Data understanding:** mempelajari struktur riwayat gacha (id, jenis banner, nama item, rank, waktu), aturan banner, dan pola laju S-Rank per pity pada data asli.
3. **Data preparation:** membersihkan data, menghitung pity dan status guaranteed, membuang data yang tidak lengkap, dan membentuk fitur.
4. **Modeling:** memilih hyperparameter dengan grid search, melatih enam algoritma klasifikasi, dan menyusun model Markov Chain sebagai acuan teori.
5. **Evaluation:** membandingkan model dengan cross-validation dan uji signifikansi, lalu menguji pada data pemain asli.
6. **Deployment:** menerapkan model ke aplikasi web Streamlit.

3.2 Sumber Data

Tabel 3.1 Sumber data penelitian

Sumber	Keterangan	Kegunaan
Simulasi Monte Carlo	400 akun virtual, 80–700 pull per banner, dibangkitkan dari model hazard (seed 42); ±314 ribu pull	Data latih dan uji utama
Simulasi untuk grid search	100 akun virtual tambahan (seed 43); ±77 ribu baris	Pemilihan hyperparameter, tidak dipakai untuk evaluasi
Data asli pemain	3 akun (anggota kelompok dan teman), 1.751 pull, 28 S-Rank; anonim tanpa UID, nama item, dan waktu	Uji model simulasi, dan eksperimen latih–uji dengan data asli saja

Simulasi dipakai karena riwayat satu akun hanya berisi ratusan pull dengan sedikit S-Rank, sehingga tidak cukup untuk melatih model. Namun, simulasi dibangkitkan dari model Markov yang memuat estimasi soft pity, sehingga model yang dilatih dengan simulasi pada dasarnya mempelajari ulang asumsi tersebut. Karena itu data asli memegang peran penting: menguji apakah model dari simulasi berlaku untuk pemain sungguhan, sekaligus memeriksa pola soft pity langsung dari data.

Selain itu, sebagai eksperimen kedua, keenam algoritma juga akan dilatih dan diuji **hanya dengan data asli** (1.425 baris setelah pembersihan). Eksperimen ini memastikan bahwa hasil perbandingan tidak bergantung pada data simulasi, dan jumlah barisnya sudah memenuhi ketentuan dataset tabel minimal 500–1.000 baris.

Riwayat asli diambil dengan tiga cara yang disediakan aplikasi: (1) skrip PowerShell yang membaca URL riwayat dari *cache* permainan di PC, (2) unggah berkas UIGF, dan (3) input manual. Skrip hanya membaca berkas di komputer pemain dan tidak mengirim data ke mana pun; *authkey* hanya dipakai sekali dan tidak disimpan.

3.3 Pra-pemrosesan Data

- Hanya pull dari banner Agent dan W-Engine terbatas yang diambil.
- Data duplikat berdasarkan kolom id dibuang, lalu data diurutkan kronologis per akun dan per banner.

- Pity dan status guaranteed dihitung ulang dari urutan pull. Kalah 50/50 ditentukan dari daftar S-Rank standar.
- **Segmen pertama** (pull sebelum S-Rank pertama yang tercatat) dibuang, karena server hanya menyimpan riwayat beberapa bulan terakhir sehingga pity pada segmen ini bisa tidak lengkap.
- Riwayat yang melebihi hard pity tanpa S-Rank ditolak karena tidak masuk akal.

3.4 Fitur dan Target

Setiap baris data adalah satu titik keputusan **sebelum** sebuah pull, sehingga tidak ada informasi masa depan yang bocor ke model.

Tabel 3.2 Fitur yang digunakan

Fitur	Keterangan
pity	jumlah pull sejak S-Rank terakhir (0 sampai hard pity − 1)
pity_ratio	pity dibagi hard pity, agar dua banner sebanding
guaranteed	1 jika S-Rank berikutnya pasti rate-up, 0 jika tidak
is_wengine	1 untuk banner W-Engine, 0 untuk banner Agent

Dua tugas klasifikasi biner didefinisikan:

- **Tugas A:** apakah pull berikutnya menghasilkan S-Rank? (kelas positif ±1,7%)
- **Tugas B:** apakah pemain mendapat minimal satu S-Rank dalam 10 pull berikutnya (satu kali *ten-pull*)? (kelas positif ±16,6%)

3.5 Pemodelan

Setiap algoritma memiliki pengaturan tetap dan beberapa *hyperparameter* yang akan dipilih dengan *grid search* (Tabel 3.3). Nilai pertama pada setiap ruang pencarian adalah pengaturan awal sebelum grid search, sehingga jika beberapa kombinasi memperoleh skor yang sama persis, pengaturan awal dipertahankan.

Tabel 3.3 Algoritma, pengaturan tetap, dan ruang pencarian grid search

Algoritma	Pengaturan tetap	Ruang pencarian grid search
Naive Bayes	Gaussian Naive Bayes	var_smoothing ∈ {10^{-9}; 10^{-6}; 10^{-3}; 0,01; 0,1; 1}
KNN	normalisasi StandardScaler, jarak Euclidean	k ∈ {101; 5; 15; 31; 51; 201}
Logistic Regression	normalisasi StandardScaler, max_iter 2000	C ∈ {1; 0,01; 0,1; 10}
Decision Tree	indeks Gini	max_depth ∈ {6; 3; 4; 8}; min_samples_leaf ∈ {50; 20; 100}
Random Forest	150 pohon	max_depth ∈ {8; 6; 12}; min_samples_leaf ∈ {20; 10; 50}
XGBoost	300 pohon, subsample 0,8	max_depth ∈ {4; 3; 6}; learning_rate ∈ {0,05; 0,1}
Pembanding: Baseline	selalu menebak "tidak dapat S-Rank" (DummyClassifier)	–
Pembanding: Markov Chain	peluang teoretis dari model hazard (Subbab 2.4); pada data simulasi menjadi batas atas akurasi karena data dibangkitkan dari model yang sama	–

3.6 Skenario Evaluasi

- Data dibagi **per akun**, bukan per pull, karena pull berurutan dari akun yang sama saling berkaitan. Pembagian per pull akan membocorkan informasi antara data latih dan data uji.
- **Grid search** pada data simulasi: hyperparameter akan dipilih memakai 100 akun simulasi tambahan (seed 43) yang tidak dipakai untuk evaluasi, dengan GroupKFold 4-fold per akun dan metrik accuracy. Pengaturan terpilih lalu dipakai untuk semua evaluasi simulasi dan model di aplikasi.
- **GroupKFold 5-fold cross-validation** dipakai untuk membandingkan model secara stabil (rata-rata ± simpangan baku).
- **Uji signifikansi:** akurasi setiap algoritma pada kelima fold dibandingkan dengan algoritma berakurasi tertinggi memakai uji t berpasangan terkoreksi (Subbab 2.7, $\alpha = 0{,}05$).
- **GroupShuffleSplit 80/20** dipakai untuk model akhir yang disimpan ke aplikasi.

- Model akhir diuji ulang pada **data asli 3 akun** yang tidak pernah dipakai saat pelatihan.
- **Eksperimen kedua:** keenam algoritma dilatih dan diuji hanya dengan data asli (1.425 baris) memakai GroupKFold 5-fold yang dibagi per **siklus pity** (25 siklus). Pembagian per siklus dipakai karena data asli hanya berasal dari 3 akun, dan siklus pity saling bebas karena peluang selalu kembali ke awal setelah mendapat S-Rank. Hyperparameter pada eksperimen ini dipilih dengan *nested cross-validation*: grid search dijalankan ulang di dalam data latih setiap fold (GroupKFold 4-fold per siklus), sehingga data uji tidak pernah ikut memilih pengaturan.
- **Analisis pola soft pity:** laju S-Rank per rentang pull pada data asli dibandingkan dengan peluang model Markov, dan aturan Decision Tree yang dilatih dengan data asli diperiksa untuk melihat di pity berapa model memisahkan data.
- Prediksi kelas memakai ambang peluang 0,5. *Oversampling* (misalnya SMOTE) tidak dipakai karena tujuan utamanya adalah angka peluang yang tepat, sedangkan menambah data buatan untuk kelas minoritas akan menggeser peluang tersebut. Ketidakseimbangan kelas ditangani dengan memilih metrik yang sesuai.
- Metrik: Accuracy (metrik utama), Balanced Accuracy, Precision, Recall, F1-score, ROC-AUC, Log Loss, dan Brier Score.

3.7 Rancangan Aplikasi

Aplikasi web dibangun dengan Streamlit, menggunakan tema gelap, dan dapat diakses dari ponsel maupun komputer. Halaman yang tersedia ditunjukkan pada Tabel 3.4.

Tabel 3.4 Halaman aplikasi

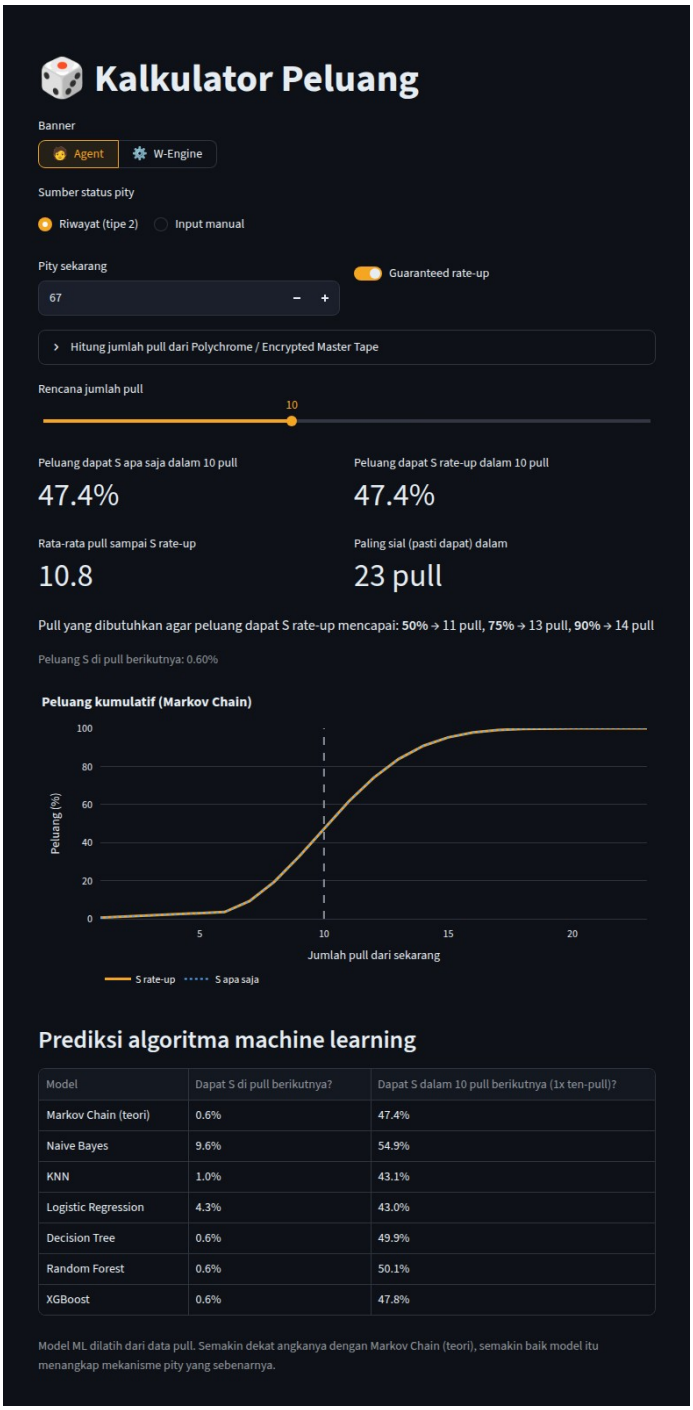
Halaman	Fungsi
Beranda	penjelasan aplikasi dan aturan banner
Import Data	tiga cara input: URL dari PowerShell, unggah UIGF, input manual; tersedia data contoh
Kalkulator Peluang	peluang S-Rank dan S-Rank rate-up dalam N pull (Markov Chain) dan prediksi 6 algoritma
Statistik Riwayat	pity setiap S-Rank, rata-rata pity, menang/kalah 50/50, pity saat ini
Perbandingan Algoritma	tabel metrik, cross-validation, uji signifikansi, hasil grid search, latih–uji data asli, pola soft pity, kurva peluang, confusion matrix, feature importance
Panduan & FAQ	cara menjalankan skrip PowerShell dan tanya jawab

3.8 Alat dan Bahan

- Bahasa pemrograman Python 3.12 dengan pustaka pandas, NumPy, SciPy, scikit-learn 1.9, XGBoost 3.2, dan Plotly.
- Streamlit 1.64 dan Streamlit Community Cloud untuk *deployment*.
- Pytest dan GitHub Actions untuk pengujian otomatis (106 kasus uji).
- Windows PowerShell untuk skrip pengambil URL riwayat.
- GitHub untuk penyimpanan kode sumber.

3.9 Hasil Awal (Prototipe)

Prototipe aplikasi telah berjalan dan dapat diakses di https://project-data-mining-zzz.streamlit.app. Riwayat tiga akun asli berhasil diambil, dan pity serta status guaranteed yang dihitung aplikasi telah dicocokkan dengan tampilan di dalam permainan dan hasilnya sama. Pada uji awal tugas B, model berbasis pohon dan KNN mencapai akurasi ±91% pada data simulasi dan ±92% pada data asli. Eksperimen latih dan uji dengan data asli saja (1.425 baris) juga memberikan hasil serupa, dengan Random Forest sebagai yang tertinggi (±92,4%). Uji coba grid search menunjukkan bahwa Naive Bayes membaik cukup besar setelah hyperparameter-nya disesuaikan, sedangkan uji signifikansi awal menunjukkan bahwa selisih antaralgoritma pada data asli yang masih sedikit belum bermakna secara statistik. Analisis awal data asli juga memperlihatkan lonjakan laju S-Rank mulai pull ke-74 pada banner Agent (dari di bawah 1% menjadi 14,3%) dan sekitar pull ke-65 pada banner W-Engine, sesuai dengan estimasi soft pity yang dipakai.



Gambar 3.2 Prototipe halaman Kalkulator Peluang

3.10 Jadwal Kegiatan

Tabel 3.5 Jadwal kegiatan proyek

Kegiatan	Minggu 1	Minggu 2	Minggu 3	Minggu 4
Studi literatur dan pemahaman aturan gacha	✓			
Pengumpulan data dan pembuatan simulasi	✓	✓		
Pra-pemrosesan dan pembentukan fitur		✓		
Pemodelan 6 algoritma dan Markov Chain		✓	✓	
Evaluasi dan uji data asli			✓	
Deployment aplikasi Streamlit			✓	✓
Penyusunan laporan dan presentasi				✓

3.11 Pembagian Tugas

Tabel 3.6 Pembagian tugas anggota kelompok

NIM	Nama	Tugas
A11.2024.16004	Syafiq Yahya	Ketua kelompok, presentasi hasil
A11.2024.15851	Dafi Hauzan A.H	Pengembang aplikasi, pengumpulan dan pengujian data
A11.2024.15842	Gastiadirrijal Rafi M	Penyusun slide presentasi
A11.2024.15826	Zabrina Miftah Z	Penyusun proposal dan laporan
A11.2024.15804	Rayya Hasya Tamimi	Penyusun proposal dan laporan

DAFTAR PUSTAKA

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review, 78*(1), 1–3.

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).

Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory, 13*(1), 21–27.

Han, J., Kamber, M., & Pei, J. (2012). *Data mining: Concepts and techniques* (3rd ed.). Morgan Kaufmann.

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering, 21*(9), 1263–1284.

Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3rd ed.). Wiley.

HoYoverse. (2024). *Zenless Zone Zero* [Permainan video]. Menu detail Signal Search: peluang dan aturan banner.

Nadeau, C., & Bengio, Y. (2003). Inference for the generalization error. *Machine Learning, 52*(3), 239–281.

Norris, J. R. (1997). *Markov chains*. Cambridge University Press.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning, 1*(1), 81–106.

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE, 10*(3), e0118432.

Streamlit Inc. (2024). *Streamlit documentation*. https://docs.streamlit.io

UIGF Organization. (2024). *Uniformed Interchangeable GachaLog Format standard v4*. https://uigf.org

Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics, 7*, 91.